

Computer Vision
2025/2026 – 2nd Semester

Practical Assignment
Development of a Computer Vision Application

Introduction

This assignment consists in the conceptualization and implementation of a prototype of an application for Computer Vision.

Each group must select/propose a work that involves the development of a Computer Vision solution using OpenCV. The use of tasks/problems resulting from work developed by students in other curricular units is encouraged.

The work must be carried out in groups of 2 elements.

The assignment will include two presentations. Presenters must use careful language, speak clearly, and are expected to not exceed available time. We advise students to prepare and train the presentation on advance.

Mid-term presentation (10 + 5 min)

In classes on 04-07/05. Each group should bring a simple presentation (< 10 min) and the conceptual model they propose to discuss with Prof. and colleagues in class.

In this presentation, it is expected that students present:

- An analysis of the problem, users, context of use and main questions that the application should solve;
- A proposal and justification for the Computer Vision algorithms and techniques they plan to use;
- Some early developments/experiments/implementations.

For the mid-term presentation, each group will have to deliver the presentation in pdf format.

Final presentation (15 + 5 min)

The final presentation, whose duration should not exceed 20 minutes (15+5 min questions), will be performed by the group members in the class on 01/06-28/05.

For the final presentation, each group will have to deliver the presentation, the produced code and a demo video (30 to 60 seconds) and a sober presentation of their work as well as a.

The final presentation must also include the indication of the **percentage of work** of **each student** to the assignment.

Delivery

Each group must deliver through e-learning by the delivery date:

- presentation slides (mid-term)
- presentation slides + code developed + illustrative video.

Use the following rule for file names: TPnnmec1+nmec2, where n is the class number and the nmec1 and nmec2 are the student numbers of the two elements of the group. For example: a group from the

TP1 class that has developed the AppXYZ application and is made up of students with nmecs 44444 and 55555 must submit a file with the name: TP1-AppXYZ-44444+55555

Warning

The developed application must be original. In addition to the files distributed in the practical classes, some code or existing libraries may be used (occasionally), but this fact must be clearly referenced in the code/presentation and report and discussed beforehand with the teacher.

Works that use a large volume of non-original code, without explicitly referencing it, will be void.

The programs created must be available for use, either through a code repository (github, gitlab) or zip file, indicating any non-original code or library used. Instructions for downloading or using dependencies, APIs or commands necessary for its operation must be explicit in a README file.

Themes

Students are encouraged to choose and propose topics of their own interest (from work developed in other curricular units, etc...), however, we suggest proposals addressing some of the main topics of the course.

The work should be developed gradually from simpler to more complex solutions.

As a rule of thumb students may use the following indication:

- **Level 1:** Use of a single solution based on existing datasets [up to 14]
- **Level 2:** Acquisition of new datasets with provided or existing cameras/sensors, test, evaluation and comparison of different approaches/solutions [up to 18]
- **Level 3:** Real time demo working on the presentation day or videos showing the robustness of the solution in different scenarios.

Automatic Correction of Multiple-Choice Exams Design a computer vision-based system, using OpenCV, to automatically correct multiple-choice exam answer sheets. The system will process scanned or photographed exam sheets, detect the answer table layout, identify marked options for each question, and compare them with a predefined answer key to compute the final score.	
Stereogram Reconstruction involves developing a program that can reconstruct hidden 3D content from stereograms (single-image or autostereograms). The system will analyse the repeating patterns in the stereogram, compute depth information from the displacement of pattern elements, and generate a 3D depth map or a visually interpretable reconstruction of the hidden shape.	
Puzzle solving program Develop a puzzle-solving program using OpenCV and classical computer vision techniques. The system will take an image of a puzzle (such as a jigsaw, sliding puzzle, or grid-based logic puzzle), detect and segment its individual components, analyse their visual features (shape, edges, colours, or patterns), and infer the correct arrangement or solution.	

3D Reconstruction Using Depth Sensors (Azure Kinect / Leica) Study and build a 3D reconstruction pipeline using data acquired from depth-sensing devices such as the Azure Kinect or a Leica 3D scanning system. Students will capture depth and/or point cloud data from real-world scenes, preprocess and align multiple views, and reconstruct a coherent 3D model. The objective is to understand the principles of depth-based 3D reconstruction and to develop a practical system capable of producing consistent 3D models of real environments or objects.	
Webcam Gesture-Controlled Game Using Computer Vision Develop a gesture-controlled game using a standard webcam and computer vision techniques. The system will capture real-time video input, detect and track the player's hand or body gestures, and map these gestures to in-game actions (e.g., moving a paddle, jumping, or controlling direction) in games such as Pong, Breakout, or Flappy Bird.	
Real-World "AR Whack-a-Mole" Using Computer Vision Create a real-world, vision-based augmented reality Whack-a-Mole game using a camera system. Virtual targets are overlaid onto a physical surface such as a table or wall, and the player interacts with them by hitting the targets using their hand or a physical object. The system will detect the interaction area, track the player's hand or tool in real time, and determine successful hits based on spatial and temporal alignment with the virtual targets.	
Snooker Helper Using Computer Vision Develop a computer vision-based snooker helper system for a miniature toy snooker table. Using a camera positioned above or at an angle, the system will detect the table boundaries, pockets, cue ball, and object balls, estimate their positions, and analyze possible shots. The program may provide visual guidance such as recommended shot directions, collision paths, or pocketing probabilities overlaid on the live video feed.	
Augmented Card Game Using Computer Vision Develop an augmented card game that combines physical playing cards with digital game logic using computer vision. A camera system is used to detect and recognize cards placed on a table, track their positions and orientations, and overlay virtual elements such as scores, animations, or game effects in real time. (Examples of possible cards: Yu-Gi-Oh, Magic The Gathering, Pokemon, Exploding Kittens).	
Augmented Reality Board and Card Games Using Computer Vision Creating an augmented reality board—such as Tic Tac Toe, Checkers, ...—using computer vision techniques. A camera observes the physical game board or cards, detects and tracks game elements, and interprets player moves in real time. The system augments the physical game with	

digital feedback, such as move validation, score tracking, hints, or animated overlays aligned with the real-world layout.	
Camera-Based Puzzle Tetris Using Computer Vision Develop a camera-based Tetris-style puzzle game controlled through hand height or gestures. A camera captures real-time video of the player, and computer vision techniques are used to detect and track the player's hand position, height, or specific gestures to control the falling blocks (e.g., horizontal movement, rotation, and drop speed).	
Edge-Preserving Smoothing for Video Cartoonization Transform real-time or recorded video into a cartoon-style game world using edge-preserving image processing techniques. The system will apply filters that smooth color regions while preserving strong edges, followed by stylization steps such as edge enhancement, color quantization, and contrast adjustment.	
Vision-Based Maze Solver Develop a vision-based maze solver that takes an image of a physical maze and automatically computes a solution path. The system uses a camera to capture the maze, applies image preprocessing and morphological operations (e.g., thresholding, erosion, dilation) to clean and enhance the maze structure, and then applies a pathfinding algorithm (such as BFS, DFS, or A*) to find a solution from start to finish.	
Camera-Based "Dot-to-Dot" Game Using Computer Vision Create a real-time dot-to-dot game where players connect points on a physical surface, such as paper or objects, while the system detects the points and overlays connecting lines digitally. Using a camera, the system captures the scene, detects marked points, tracks their order of connection, and renders lines or shapes in real time.	
Explore other vision problems / Datasets http://homepages.inf.ed.ac.uk/rbf/CVonline/Imagedbase.htm	
*** Students can propose other assignments in line with their interests, master topic or other topics	

Autor: [Paulo Dias](#)